

CampusEats Integration Solutions

Combined Integration Documentation — Questions 1, 4, 5, and 6

QUESTION 1

Pick the partner edge and justify SOAP

Task Requirement: Choose one external partner CampusEats must integrate — a payment gateway, a logistics/delivery partner, or an SMS gateway. In 3–4 sentences, explain why that edge is SOAP while the rest of CampusEats is REST (strong contract, message-level security, transactions).

Submit: A short “Context” section in integration.pdf naming the partner and the one operation you’ll integrate.

CampusEats - Integration Context

Context

Partner Edge: External Payment Gateway

Operation: charge

CampusEats will integrate with an external Payment Gateway using SOAP for the charge operation, while the other CampusEats services communicate using REST. SOAP is appropriate at this external partner edge because payment processing benefits from a strong, formally defined contract, message-level security, and reliable transaction handling for financial operations. The Payments service will hide the external gateway implementation and expose its own contract to Orders, maintaining the independence and loose coupling described in the CampusEats architecture. The gateway can therefore be replaced without requiring changes to the Orders service, since Orders only depends on the Payments contract.

QUESTION 4

Show the HTTP binding

Task Requirement: Show the HTTP POST that carries the request: method, Host, Content-Type, the SOAPAction value from your binding, and the endpoint URL from your service/port.

Submit: The HTTP request block in integration.pdf.

CampusEats HTTP SOAP Binding & POST Request

This document is based on the Campus Eats repository and its `partner.wsdl` / `soap-request.xml` files. The WSDL defines a SOAP 1.1 HTTP binding for the charge operation.

1. HTTP Binding

Item	Value
Binding	PaymentSoapBinding
Style	document
Transport	http://schemas.xmlsoap.org/soap/http
Operation	charge
SOAP body	literal
SOAPAction	http://payments.campuseats.example.com/charge
Service	CampusEats PaymentService
Port	PaymentPort
Endpoint	https://payments.example.com/campuseats/PaymentService

2. HTTP POST that Carries the SOAP Request

The endpoint host is taken from the WSDL `soap:address`. Because the WSDL uses the SOAP 1.1 HTTP transport namespace, the request uses the SOAPAction HTTP header and a `text/xml` content type.

```
POST /campuseats/PaymentService HTTP/1.1
Host: payments.example.com
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://payments.campuseats.example.com/charge"

<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="http://payments.campuseats.example.com/">
  <soap:Header>
    <pay:Credentials>
      <pay:merchantId>CAMPUS_EATS</pay:merchantId>
      <pay:apiKey>demo-key-12345</pay:apiKey>
    </pay:Credentials>
  </soap:Header>
  <soap:Body>
    <pay:charge>
      <pay:orderId>CE-1001</pay:orderId>
      <pay:amount>499.00</pay:amount>
      <pay:currency>INR</pay:currency>
      <pay:cardToken>tok_demo_123456</pay:cardToken>
    </pay:charge>
  </soap:Body>
</soap:Envelope>
```

3. Required Values

Field	Value
Method	POST
Host	payments.example.com
Content-Type	text/xml; charset=utf-8
SOAPAction	"http://payments.campuseats.example.com/charge"
Endpoint URL	https://payments.example.com/campuseats/PaymentService

4. Source Mapping

partner.wsdl: PaymentSoapBinding, document style, SOAP/HTTP transport, charge SOAPAction, PaymentPort, and the payment endpoint.

soap-request.xml: SOAP envelope, Credentials header, and charge request body.

Note: The repository uses example domains (payments.example.com), so this endpoint is a documented/example endpoint rather than evidence of a live production payment server.

QUESTION 5

Describe discovery (modern form)

Task Requirement: Explain how CampusEats would obtain this WSDL — a direct URL, or a lookup — and write a one-record catalogue / registry entry: business, service, endpoint, and the WSDL it speaks (a tModel-style pointer). No live UDDI server is required.

Submit: A “Discovery” section with the registry entry in integration.pdf.

CampusEats Integration - Discovery

Discovery (modern form)

Modern service discovery can be represented without deploying a live UDDI server. A consumer can obtain the service contract either by using a direct, well-known WSDL URL or by looking up a registry/catalogue record that points to the WSDL. The catalogue acts as the modern equivalent of a UDDI business/service entry, while the WSDL remains the authoritative contract for the SOAP service.

Direct URL approach: CampusEats could publish or distribute the WSDL at a stable URL. For this repository, the WSDL file is `partner.wsdl`. A consumer can retrieve that document directly from the repository's raw-file URL and then read the service, port, binding, SOAPAction, and endpoint information from it. The WSDL declares the service as *CampusEats PaymentService*, the port as *PaymentPort*, and the endpoint as `https://payments.example.com/campuseats/PaymentService`.

Lookup approach: Instead of knowing the WSDL URL in advance, an integration catalogue or registry could store one record for the CampusEats payment service. A consumer searches by business and service name, receives the endpoint and a tModel-style pointer to the WSDL, then retrieves the WSDL and generates/configures its SOAP client from that contract. No live UDDI infrastructure is needed for this assignment; the following is a one-record conceptual registry entry.

One-record catalogue / registry entry

Field	Registry value
Business	CampusEats
Service	CampusEats PaymentService
Endpoint	<code>https://payments.example.com/campuseats/PaymentService</code>
WSDL / tModel-style pointer	<code>https://raw.githubusercontent.com/udaygupta9081/campuseats/main/partner.wsdl</code>

Registry entry as a compact record

```
{
  "business": "CampusEats",
  "service": "CampusEats PaymentService",
  "endpoint": "https://payments.example.com/campuseats/PaymentService",
  "tModel": {
    "name": "CampusEats Payment SOAP WSDL",
    "wsdl": "https://raw.githubusercontent.com/udaygupta9081/campuseats/main/partner.wsdl"
  }
}
```

How the lookup works

1. The client searches the catalogue for the business **CampusEats** and service **CampusEats PaymentService**.
2. The catalogue returns the service endpoint and the tModel-style WSDL pointer.
3. The client retrieves `partner.wsdl` from that pointer.
4. From the WSDL, the client learns the SOAP binding, the charge operation, the SOAPAction, and the service/port endpoint, and can construct the SOAP request.

Repository evidence

The repository WSDL defines `PaymentSoapBinding` as a document-style SOAP binding over HTTP, defines the `charge` operation and its SOAPAction, and binds `PaymentPort` to the payment endpoint. These declarations are the contract referenced by the catalogue entry.

QUESTION 6

Map the fault into your own contract

Task Requirement: Show how the partner's fault becomes the error your Assignment 2 contract promised (e.g. the partner's `card_declined` → your `placeOrder` "payment declined"). The partner's vocabulary must not leak to your students.

Submit: A short "Fault mapping" note in `integration.pdf`.

CampusEats Integration - Fault Mapping

Fault mapping

The partner's fault vocabulary is translated at the integration boundary so it does not leak into the CampusEats student-facing contract. If the partner returns a SOAP fault indicating `card_declined`, the CampusEats adapter catches that partner-specific fault and maps it to the Assignment 2 `placeOrder` contract error `payment declined`.

Partner response	CampusEats contract response
Partner SOAP fault code/reason: <code>card_declined</code>	<code>placeOrder</code> payment declined
Partner-specific fault details	Not exposed to the student/client contract
Adapter responsibility	Catch, translate, and return the contract-defined payment error

Boundary rule: The partner's names such as `card_declined`, partner fault codes, and partner SOAP fault structure are implementation details of the adapter. The only error visible through the CampusEats contract is the agreed `payment declined` error. This keeps the student-facing API independent of the partner's vocabulary.